Claude Code for Academic Researchers

A Methods Workshop in AI-Assisted Research

Brady Allardice

UPF & IBEI

About Me

Brady Allardice
PhD Candidate, Political & Social Sciences
UPF / IBEI, Barcelona

Research: Political economy of technological change

- Automation, AI, and political behavior
- LLM-based text classification pipelines

Background:

- M.Res. Political Science (UPF)
- M.Sc. Economics (Barcelona School of Economics)
- Former data scientist

Why this course:

- Daily user of Claude Code for academic research
- Use it across the full pipeline: data cleaning, analysis, robustness checks, code auditing, replication, writing, reviewing
- These tools can dramatically expand what a researcher can accomplish, if used with the right workflows

brady.allardice@upf.edu
github.com/bradyall

Introductions

1. Who are you? (name, department, research area)
2. What does your daily workflow look like? (tools, languages, data)
3. How do you currently use AI, if at all?
4. What do you hope to learn from this course?

Course Roadmap

Session	Focus
1	Introduction: what Claude Code is, what you can do with it, and how it works
2	Research pipelines: from data to results — cleaning, estimation, robustness, LaTeX tables
3	Git and LaTeX in VS Code: version control and the integrated research environment
4	Advanced Claude Code: skills and MCPs (Zotero, GitHub)
5	Subagents, hooks, and the final project

5 sessions $\times$ 2 hours | 60–70% hands-on | Pair programming throughout

The Central Idea

You are the principal investigator. Claude Code is the research assistant.
You direct the work, make analytical decisions, and verify every result.
Claude Code handles the mechanical execution.

The question is never "can Claude Code do this." It almost always can.

The question is: **what is the most productive and reliable way to get this done?**

By the end of this course, you will be able to:

- Automate data cleaning, robustness checks, code auditing, and documentation
- Choose the right execution mode for each task
- Verify output against known properties of your data
- Recognize when Claude Code is failing and intervene effectively

Module 1

What This Is and What You Can Do With It

Demo + Exercise

How We'll Work: Your Own Workspace

The course materials live in one shared repo, organized as self-contained session folders:

```
claude-code-workshop/
  session_1/ <- today's materials live here
  session_2/
  session_3/
  ...
```

The workflow, every session: copy that session's folder out of the repo into your own workspace, and work there. The shared repo stays read-only for you.

Why

I can push new material anytime without breaking your work. You can edit, break, delete, and redo without worrying about the shared state. No merge conflicts.

The Start-of-Session Ritual

Do this at the start of every session. Today is the first time:

```
# 1. Get the course repo (first time: clone it)
git clone https://github.com/bradyallardice/claude-code-workshop.git
cd claude-code-workshop

# 2. Make your workspace and copy today's folder into it
mkdir -p ~/my_workspace
cp -r session_1/ ~/my_workspace/session_1/

# 3. Open your copy in VS Code and work there
cd ~/my_workspace/session_1 && code .
```

Already have the repo from pre-work? Run `git pull` inside it instead of cloning, then continue with Steps 2–3.

We do it together now, live. Everyone ends up in the same clean state before we move on.

Setup Check

Open the terminal in VS Code (or your standalone terminal) and run:

1. Verify Node.js: `node --version`
2. Verify Claude Code: `claude --version`
3. Launch Claude Code: `claude`

You should see Claude Code's interface appear in your terminal.

Try typing a simple message: `What files are in this directory?`

If you completed pre-work: help your neighbor.
If you're stuck: pair with someone who is set up. We'll troubleshoot at break.

Live Demo: The Data

Dataset 1: County Presidential Election Returns

- Source: MIT Election Data + Science Lab
- County-level vote totals by candidate and party, 2000–2024
- ∼94,000 rows, one row per county × year × candidate
- Variables: FIPS codes, candidate names, party, vote counts, vote mode

Dataset 2: IPUMS American Community Survey Microdata

- Source: IPUMS USA (University of Minnesota)
- Person-level survey records with demographic, economic, and geographic variables
- ∼15 million rows per year, one row per person
- Variables: age, education, employment, income, race, poverty status, county FIPS

Live Demo: The Task

Goal: Build a county-year panel joining election results with demographics, and visualize the outputs.

This requires:

1. Reshape election data to one row per county-year with vote shares
2. Aggregate 15M person-level records to county-level means using survey weights
3. Harmonize FIPS codes across datasets (different formats)
4. Merge on county $\times$ year and validate the join
5. Create visualizations of trends over time

In Stata or R, this takes 30–45 minutes of careful coding.

What Just Happened

It wrote the code:

- Read both files
- Identified the join key
- Handled cleaning steps
- Wrote a complete script

It also:

- Understood the data structure (person-level vs. county-level, survey weights, FIPS formats)
- Ran the code and diagnosed errors
- Provided summaries of the output (row counts, variable descriptions, merge diagnostics)

But it also made a decision about our data without asking.

Both of these things are true:

1. It took 90 seconds instead of 45 minutes
2. It made analytical choices autonomously

The RA Analogy

How to Think About Claude Code

Claude Code is like a research assistant who is:

- Extremely fast
- Has read a lot
- Will **never** say "I don't know how to do that"

Instead, it will just do it wrong and hand it in.

You have all managed RAs. You know how valuable, and how dangerous, that profile is.

This course teaches you to capture the value while managing the risk.

The Coding Paradigm Spectrum: Manual Coding

Manual Coding

Autocomplete / Copilot

Web Chat

Agentic Coding

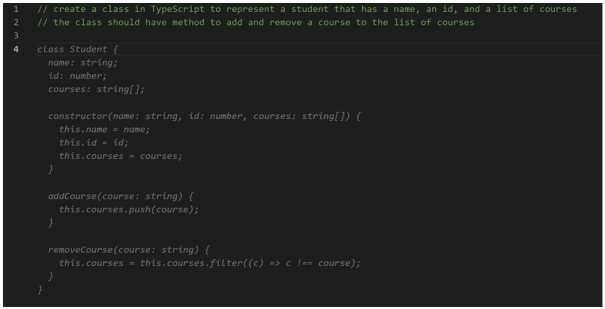
You write every line. Full control, no assistance.

This is what you do now in Stata or R.

```python
def load_tasks_to_dwas(tasks_to_dwas_file: str) -> pd.DataFrame:
    """
    Load Tasks-to-DWAs mapping file.

    Returns DataFrame with columns: O*NET-SOC Code, Task ID, DWA ID, DWA Title
    """
    if not os.path.exists(tasks_to_dwas_file):
        raise FileNotFoundError(f"Tasks-to-DWAs file not found: {tasks_to_dwas_file}")

    df = pd.read_excel(tasks_to_dwas_file)

    required_cols = ['O*NET-SOC Code', 'Task ID', 'DWA ID', 'DWA Title']
    missing = [c for c in required_cols if c not in df.columns]
    if missing:
        raise ValueError(f"Missing columns in Tasks-to-DWAs file: {missing}. Available: {list(df.columns)}")

    logger.info(f"Loaded {len(df)} task-DWA mappings ({df['Task ID'].nunique()} unique tasks, "
                f"{df['DWA ID'].nunique()} unique DWAs)")
    return df


def load_dwa_reference(dwa_reference_file: str) -> pd.DataFrame:
    """
    Load DWA Reference file with IWA (Intermediate Work Activity) context.
```

The Coding Paradigm Spectrum: Autocomplete / Copilot

- Manual Coding
- **Autocomplete / Copilot**
- Web Chat
- Agentic Coding



```
// create a class in TypeScript to represent a student that has a name, an id, and a list of courses
// the class should have method to add and remove a course to the list of courses

class Student {
  name: string;
  id: number;
  courses: string[];

  constructor(name: string, id: number, courses: string[]) {
    this.name = name;
    this.id = id;
    this.courses = courses;
  }

  addCourse(course: string) {
    this.courses.push(course);
  }

  removeCourse(course: string) {
    this.courses = this.courses.filter((c) => c !== course);
  }
}
```

AI suggests the next line as you type.

You accept or reject. The AI assists; you drive.

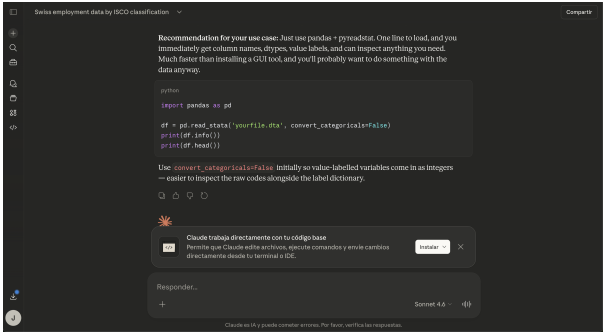
The Coding Paradigm Spectrum: Web Chat

Manual Coding

Autocomplete / Copilot

Web Chat

Agentic Coding



You describe a task; AI gives you code to copy-paste.

The AI cannot see your project.

The Coding Paradigm Spectrum: Agentic Coding

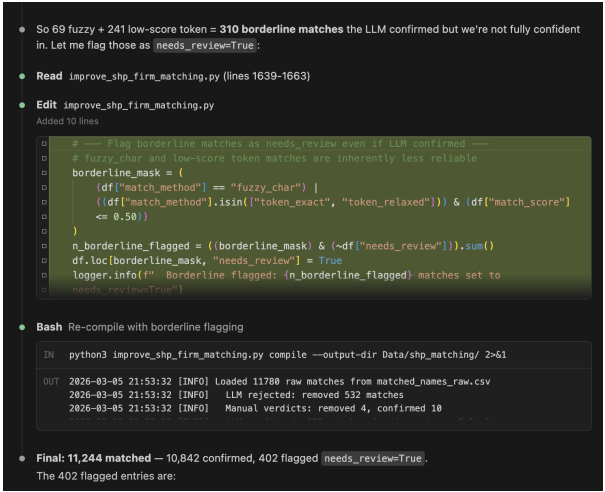
Manual Coding

Autocomplete / Copilot

Web Chat

Agentic Coding

AI reads, writes, and runs code **in your project**.



Why Agentic Coding Is Different

Paradigm	Pros	Cons
Manual Coding	Full control	Slow, everything by hand
Autocomplete / Copilot	You control logic and structure	Still line-by-line
Web Chat	Can handle complex tasks	Copy-paste; AI misses context
Agentic Coding	**Fastest; works on your project**	**Can act autonomously**

The Key Insight

Agentic coding is the first paradigm where the tool **interacts with your project directly and autonomously**. That is what makes it so productive, and why learning to use it well matters.

These principles apply to Claude Code, Cursor, Codex, Windsurf, or whatever comes next.

What Claude Code Can Do for Academic Research

Data Work

- Clean, reshape, and merge datasets
- Construct variables from raw data
- Build panel datasets across sources
- Handle missing data, recoding, harmonization

Analysis

- Exploratory data analysis and summary statistics
- Generate robustness checks and specification variants
- Create publication-quality visualizations
- Synthesize results across specifications

Writing and Review

- Draft methods sections, data descriptions, results summaries
- Audit existing scripts for silent errors
- Review code against your methods section
- Write documentation, READMEs, replication packages

Research Infrastructure

- Process text corpora at scale (NLP, classification)
- Web scraping and API data collection
- Crosswalk and linkage construction
- Refactor and optimize slow pipelines

A Tool Built for Software Engineers

Claude Code was designed for professional developers. Their world is different from ours.

Software Engineering

- "Works" = **runs correctly**. Clear success criteria: tests pass, feature ships, users are happy
- Code is the product
- Errors are caught by automated tests
- Speed and iteration are paramount

Academic Research

- "Works" = **???**. Code that runs without errors can produce completely wrong results. Output that looks plausible can be based on the wrong sample, wrong specification, or wrong variable
- Code is the *instrument*; the paper is the product
- Errors are caught by *you*, or by Reviewer 2
- Correctness matters more than speed

The defaults, instincts, and shortcuts built into these tools reflect engineering priorities.
Using them well for research means overriding some of those defaults.

Do You Own It?

Pick any result your code produced. Without looking at the code:
what got dropped, what got merged, and why?

If you can't answer that in 30 seconds, you don't own it.

Before you delegate:

- What would wrong look like? Can I describe a specific failure mode before I run this?

After you delegate:

- Can I describe what this code does without reading it?
- Can I state what decisions shaped the output and why?

You didn't write it. That's fine.
But every decision in it is yours. **Can you justify it as if you chose it yourself?**

Why Verification Matters
What happens when you skip verification:

1. **Silent observation loss:** A merge silently drops 300 observations.
   The code looks clean. The error is invisible without checking row counts.
2. **Invalid transformation:** A log transformation applied to a variable containing zeros.
   No error raised, wrong results.
3. **Incorrect panel operations:** A lagged variable computed without sorting by time within panels.

These are the kinds of errors that end up in published work.

The Control Principle

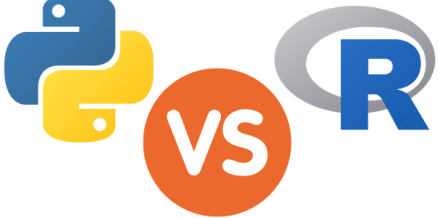
You Are the PI. Claude Code Is the RA.

The question is not "can Claude Code do this" but "what is the best way to get this done reliably."

If you cannot verify a result, you should not delegate the task.

Why Python

- Claude Code is substantially better at Python than Stata or R
- The principles (task decomposition, verification, control) transfer to any language
- Python is increasingly where the tools and ecosystem are for computational research

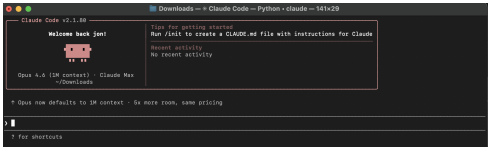


Where Does Claude Code Live?

Claude Code runs **inside your terminal**, not in a browser, not in an app.

1. You open a terminal
2. Navigate to your project: `cd my_project/`
3. Type `claude` and press Enter
4. You're now talking to Claude Code

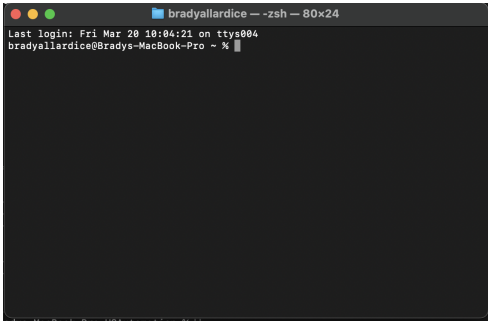
It can see your files, read your data, write scripts, and run them, all without copying and pasting.



This is what Claude Code looks like when you launch it.

What Is the Terminal?

The terminal is a text-based interface to your computer. Instead of clicking, you type commands.



Claude Code writing and running code in the terminal.

Essential commands:

```
# Where am I?
pwd

# What's in this folder?
ls

# Move into a folder
cd my_project/

# Run a Python script
python my_script.py

# Launch Claude Code
claude
```

Same Claude Code, Different Front Doors

Claude Code is **one agent**. You can reach it from several places, and it behaves the same in all of them:

Where	What it is / when to use it
Terminal (CLI)	The original. Run `claude` in any project folder — full agent, always available, great for quick jobs or tools without an editor plug-in.
VS Code / JetBrains	**Our home base for this course.** The same agent inside your editor: inline diffs, click-to-open file references, your code and Claude side by side.
Desktop app	Claude Code in its own window (Mac/Windows) instead of a terminal.
Web (claude.ai/code)	Claude Code in the browser for cloud sessions — nothing to install.

Not the same as the Claude app

The **Claude app** (claude.ai or the desktop chat) is a conversational assistant: you paste text in and copy answers out. It can't see your project, run your code, or edit files in place. **Claude Code** acts directly in your repo — reading files, editing them, and running commands for you.

Terminal Example: Editing a Document

The terminal isn't just for code. Open Claude Code in any folder and ask it to edit a document — notes, a README, a draft.

```
cd ~/my_workspace/session_1
claude
```

Then ask, in plain language:

```
> Open notes.md and tighten the Summary section
  to three bullet points. Keep my wording where
  you can, and don't touch the other sections.
```

Claude reads the file, shows a **proposed diff**, and waits for you to **approve, reject, or ask for changes** — the same review loop you'll use everywhere.

Terminal vs. VS Code

The terminal is how you'd normally interact with Claude Code — especially if you're working in **Stata**, where there isn't a tight editor integration.

But for this course, we'll be working in **VS Code**: the file you're editing, the code Claude is writing, and the conversation all live in one window.

Why VS Code?

VS Code is a free code editor with a built-in terminal.

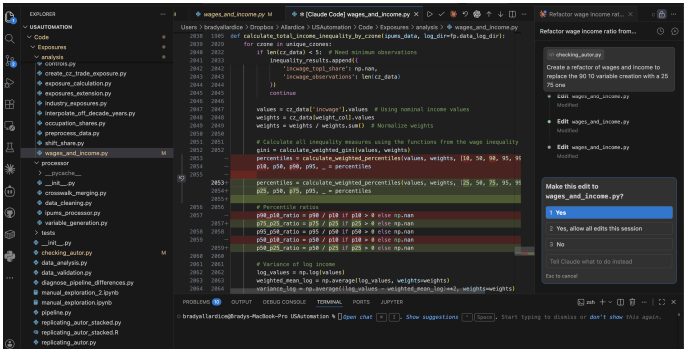
Everything in one window:

- **Left:** file explorer
- **Center:** your code
- **Right:** Claude Code

You see what Claude Code writes *as it writes it*.

Today we use three features:

1. Open a folder
2. View files in the editor
3. Talk to Claude Code

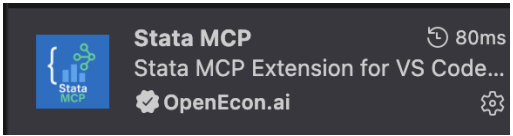


Coming from RStudio? ▸ Configure VS Code to look like RStudio

VS Code Runs Your Code Too

VS Code isn't just an editor — with the right extensions, it executes your scripts directly:

- **Python** (`.py`, `.ipynb`) — run line-by-line with Shift+Enter, or run the whole file
- **R** (`.R`, `.Rmd`) — same workflow as RStudio, same interactive console
- **Stata** (`.do`) — run do-files via the Stata extension, results in the integrated terminal



The Stata extension for VS Code.

One editor, one window: Claude writes the code, and you run it without leaving the screen.

Python Files: `.py` vs. `.ipynb`

Notebooks (`.ipynb`)

```
df_ipums_pre_exposure = pd.read_pickle(f'/Users/bradyallardice/Desktop/PhD/Projects/USAutomation/Data/Merge/IPUMS/processed_ipums_1040.pkl')
df_ipums_pre_exposure.groupby(['occ1950rj','czone'])['perwt_by_afacter'].sum().reset_index()
```

[29]

Python

	occ1950rj	czone	perwt_by_afactor
0	2.0	3700.0	62.676506
1	2.0	3800.0	37.315493
2	2.0	4200.0	9.041921
3	2.0	4301.0	12.407893
4	2.0	4702.0	12.759623
...	...	...	...
68909	970.0	39206.0	238.110368
68910	970.0	39301.0	83.274260
68911	970.0	39302.0	1712.524076
68912	970.0	39303.0	1064.010787
68913	970.0	39400.0	25848.531526

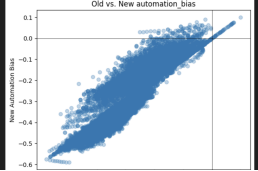
68914 rows × 3 columns

Visualizing Automation Exposure Changes

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots()
ax.scatter(df_final['automation_bias_per_worker'], df_final['automation_bias_per_worker_ffilled'], alpha=0.3)
ax.set_xlabel('Old Automation Bias')
ax.set_ylabel('New Automation Bias')
ax.axhline(0,color='k',linewidth=.5); ax.axvline(0,color='k',linewidth=.5)
ax.set_title('Old vs. New automation_bias')
plt.show()
```

[30]

Python



Scripts (`.py`)

Code > Exposures > checking_autor.py > ...

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.stats import pearsonr, spearmanr
import os
import pickle
from pathlib import Path

# Set up plotting style
plt.style.use('ggplot')
sns.set_context("talk")

import General.filepaths as fp

def load_data_files():
    """Load all intermediate data files needed for diagnostics"""
    data_files = {}

    # Try to load cz-level data
    try:
        occ_exposure_path = f'{fp.exposures_out_dir}occ_ind_4080rj_panel_cw_t85.dta'
        data_files['occ_exposure'] = pd.read_stata(occ_exposure_path)
        data_files['occ_exposure'] = data_files['occ_exposure'][['year','occ1950rj','N
        print(f"✓ Loaded occupation exposure data from {occ_exposure_path}")
        print(f"  Shape: {data_files['occ_exposure'].shape}")
    except FileNotFoundError:
        print(f"✗ Could not find occupation exposure data at {occ_exposure_path}")

    # Try to load occupation shares data
    try:
        occ_shares_path = f'{fp.exposures_out_dir}final_occ_shares_all_years_by_cz.csv
        data_files['occ_shares'] = pd.read_csv(occ_shares_path)
        print(f"✓ Loaded occupation shares data from {occ_shares_path}")
        print(f"  Shape: {data_files['occ_shares'].shape}")
    except FileNotFoundError:
        print(f"✗ Could not find occupation shares data at {occ_shares_path}")
```

Why Scripts for This Course

Notebooks (`.ipynb`)

- Cells you run one at a time
- Great for exploration and teaching
- State depends on which cells you ran and in what order
- Hard for Claude Code to work with because it cannot "click" cells

Scripts (`.py`)

- Plain text file, runs top to bottom
- Reproducible: `python my_script.py` always does the same thing
- Claude Code can read, write, and run them directly
- **This is what we use today**

The Key Difference

Claude Code operates on files, not interactive sessions. Scripts run end-to-end and produce the same result every time, exactly what you need for reproducible research.

What Changes When You Use Scripts

You can run scripts line by line in VS Code (Shift+Enter), just like a notebook. But scripts aren't really built for that.

Their strength is that they run **end-to-end**: `python my_script.py` always does the same thing, top to bottom. That is what makes them reproducible, and what Claude Code needs.

What this means for your workflow:

- More **checkpointing**: save intermediate datasets to disk so you can inspect them
- More **logging**: print row counts, shapes, and summaries as the script runs
- More **intermediate file saves**: write CSVs between pipeline stages
- Overall **less interactive**, more structured

This feels slower at first. But it produces pipelines that are reproducible, debuggable, and that Claude Code can work with directly.

Virtual Environments

What is a virtual environment?
A self-contained folder that holds a separate copy of Python and its packages. Each project gets its own environment, so packages from one project don't interfere with another.

Why you need one:

- Without it, every project shares the same packages, and updating one project can break another
- Claude Code installs packages when it needs them. You want that isolated, not touching your system Python
- Reproducibility: you know exactly which packages your project uses

Think of it like this: a virtual environment is a clean desk for each project. You only put the tools you need on it, and cleaning up one desk does not affect the others.

Setting Up a Virtual Environment

Run these commands in your terminal, from your project folder:

```
# 1. Create the environment (once per project)
python3 -m venv AIAgents

# 2. Activate it (every time you open a new terminal)
source AIAgents/bin/activate # Mac/Linux
AIAgents\Scripts\activate # Windows

# 3. Install packages
pip install pandas numpy statsmodels

# 4. Confirm it's active (you should see (AIAgents) in your prompt)
which python
```

In VS Code: once you create the `AIAgents` folder, VS Code will detect it and ask to use it as your interpreter. Say yes. After that, every terminal it opens will activate the environment automatically.

Detour: Claude Code in the Terminal

We'll live in VS Code for this course. But the terminal CLI is always there — and it's the natural fit for tools *without* an editor integration, like **Stata**.

Say you inherited a legacy `analysis.do` and want to understand and modernize it. Open a terminal in that folder and launch Claude Code:

```
cd ~/my_workspace/session_1/stata_demo
claude
```

No editor, no setup — just you, the folder, and the agent. Same Claude Code you use in VS Code.

Terminal Example: A Stata `.do` File

Once Claude Code is running, ask in plain language:

```
> Read analysis.do and explain what each block does.
  Then port it to a documented Python script using
  pandas and statsmodels. Flag anything Stata-specific
  that needs a decision from me before it translates.
```

What Claude does:

- Reads `analysis.do` and walks you through the regressions
- Writes `analysis.py`, matching the model and output
- Flags judgment calls — e.g. `xtset` panel structure, robust vs. clustered SEs — instead of guessing

You still decide. The terminal is just a faster door for a one-off job.

Exercise: Your First Commands

Pick your language. The starter script builds a county-year vote-share panel — find it in `session_1/scripts/`:

- **Python:** `exercise_build_panel.py`
- **R:** `exercise_build_panel.R`
- **Stata:** `exercise_build_panel.do`

Task 1: Ask Claude Code to explain what your script does, line by line.

Task 2: Ask it to add verification checks, documentation, and print statements.

Task 3: Ask it to change the two-party vote shares to total vote shares.

Exercise Debrief

- What did it get right?
- Anything wrong or confusing?
- That is normal and expected

What's Ahead

By the end of this course, you will have:

- Built a documented data pipeline
- Generated robustness checks
- Audited code for errors
- Learned version control for agent-assisted work

You will know which parts Claude Code should handle and which require your judgment.

And you will be able to take these workflows back to your own research.

Now that you have seen what Claude Code can do,
let us look at how it actually works.

Context, modes, and the conventions that make it reliable.

From Interactive to File-Based Workflows

What you do now (Stata / R):

- Open a session, load data into memory
- Run commands interactively, see results immediately

How Claude Code works:

- Reads and writes **files on disk**
- No persistent session with your data loaded
- Reads your files $\rightarrow$ writes/modifies scripts $\rightarrow$ optionally runs them

Practical Implication

Think in Projects, Not Sessions

You need to think in terms of:

- Project directories
- File paths
- Scripts that run end-to-end

Not interactive exploration.

The Context Window

Claude Code can only "see" a limited amount of text at once.

The context window includes:

- Your instructions
- The files it reads
- Its own responses

When a conversation gets long or a project gets large, it loses earlier context.

Why the Context Window Matters

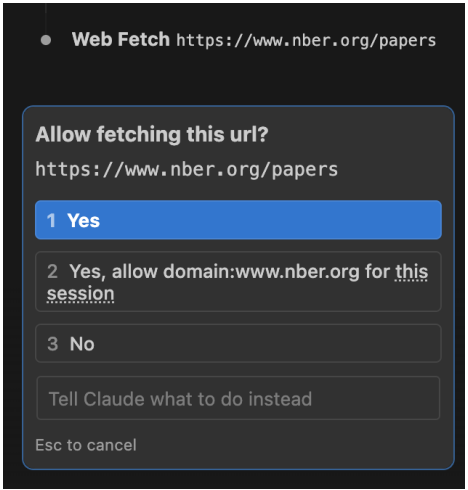
The Consistency Problem

In a complex project with many files, Claude Code may change one file in a way that is **inconsistent** with another file it can no longer see.

Errors compound when the agent loses track of earlier decisions.

We will cover strategies for managing this in Module 3.

When Claude Code Asks for Permission



Before taking actions (running code, writing files), Claude Code asks you.

Your three options:

1. **Yes**: allow this one time
2. **Yes, always allow**: allow this type of action for the rest of the session (no more prompts)
3. **No**: reject, and tell Claude Code what to do differently

"No" is not a dead end.

Use it to redirect: *"No, use a left join instead"* or *"No, don't overwrite that file, create a new one."*

Common Actions and Risk Levels

Action	Example you'll see	What it does	Risk level
`Read`	`Read scripts/clean.py`	Looks at a file	Safe
`Glob`	`Glob data/**/*.csv`	Finds files by name	Safe
`Grep`	`Grep "county_fips"`	Searches inside files	Safe
`Edit`	`Edit clean.py line 42`	Changes existing code	Review first
`Write`	`Write scripts/merge.py`	Creates a new file	Review first
`Bash`	`python scripts/clean.py`	Runs a command	Read before approving
`Bash`	`pip install pandas`	Installs a package	Read before approving
`Bash`	`rm data/old_panel.csv`	Deletes a file	Read before approving
`Bash`	`mv output.csv data/`	Moves a file	Read before approving
`Bash`	`git push`	Uploads to remote server	Read before approving

Safe to "always allow":

Reading and searching. Claude Code is just looking, not changing anything.

Always read before approving:

Bash commands can do *anything*: run code, delete files, install software. Check what it is actually running.

Protecting Sensitive Files

"Always allow reads" assumes there is nothing sensitive to read.
If your project has restricted data or IRB-protected files, you need to keep them away from Claude Code.

For now, the reliable rule is simple:

- Don't keep restricted or identifiable data inside the folder you run Claude Code in.
- Point Claude at a working copy with the sensitive files or columns removed, or keep that data in a separate location entirely.
- When in doubt, don't let it read the file — review before you approve.

Coming in Session 4

There are dedicated guardrails — ignore rules and permission denies — for hiding files from Claude. We'll set them up properly, and see where the naive version quietly fails, once you have a real project.

The Three Execution Modes

The most important configuration decision you will make.

Plan Mode: Reviews approach with you *before* acting

Default Mode: Acts, but asks permission for "significant" steps

Auto-Accept: Executes everything without asking

↓ More autonomy, less oversight ↓

How it works:

- Claude Code analyzes your request
- Proposes a detailed plan (files to read, changes to make, approach)
- Waits for your explicit approval before acting
- You review, approve, modify, or reject

When to use:
Any task where the *approach* matters, not just the output.
Data transformation, variable construction, merges, anything analytical.

Why It Matters

The plan is your checkpoint. This is where you catch "I am going to do an inner join" *before* it happens, not after 300 observations are gone.

How it works:

- Claude Code executes steps autonomously
- Pauses for actions *it* considers significant
- Uses its own judgment about when to ask

When to use:
Routine tasks in established projects where you trust the general approach.

The Risk for Academics

Claude Code's judgment about what is "significant" may not match yours.
It might pause before writing a file but **not** before silently recoding a variable.

How it works:

- Executes everything without asking
- Reads, writes, runs, and modifies freely
- Stops only on errors

When to use:

- File reformatting
- Boilerplate generation
- Documentation

When not to use:
Anything involving your data or analysis.
The speed gain is not worth the loss of checkpoints.

The Key Insight on Modes

Different Relationships with the Agent

Plan mode = you **direct** the work
Auto-accept = you **review** the work after the fact

For published research, directing is almost always safer than reviewing after the fact.

Why: Errors you *prevent* at the planning stage are cheaper than errors you *catch* in the output, or worse, errors you don't catch at all.

Live Demo: Plan Mode vs. Auto-Accept

Same task, two modes.

Watch for:

- What does a plan look like?
- How do you evaluate it?
- What does the experience feel like in each mode?

Choosing a Model

Claude Code lets you switch between three model tiers.
Each trades off **quality**, **speed**, **cost**, and **context window size**.

	Haiku 4.5	**Sonnet 4.6**	**Opus 4.6**
Context window	200k tokens	200k tokens	200k–1M tokens
Speed	Fastest	Fast	Slowest
Cost	Lowest	Moderate	Highest
Reasoning	Basic	Strong	Strongest

To switch models in Claude Code:
Type `/model` and select, or press `Shift+Tab` to toggle.

When to Use Each Model

Haiku

Fast, cheap, good enough for simple tasks.

- Reformatting files
- Renaming variables
- Generating boilerplate
- Quick questions about syntax

Sonnet

The everyday workhorse for most research tasks.

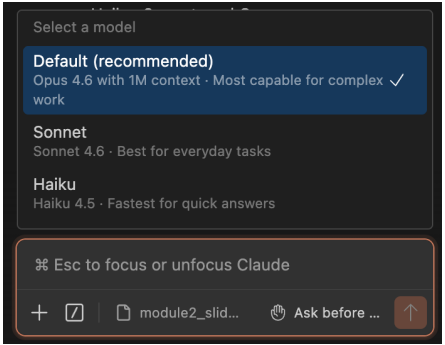
- Writing data cleaning scripts
- Building panels and merges
- Debugging code
- Drafting documentation

Opus

Maximum reasoning power; use when it matters.

- Complex multi-file refactors
- Subtle specification decisions
- Reviewing analytical code
- Large-context projects (up to 1M tokens)

Model Choice: The Research Perspective



The Key Tradeoff

Smarter models are slower and more expensive.
A wrong answer fast is worse than a right answer slow.

Rules of thumb:

- **Default to Sonnet**: best balance of quality and speed
- **Upgrade to Opus** for careful reasoning or many files
- **Drop to Haiku** for mechanical, easy-to-verify tasks

Thinking vs. Not Thinking

Beyond *which* model, you control *how hard* it works before it answers.

Not thinking (default)

The model responds directly. Fast and cheap.

- Reformatting, renaming
- Simple, well-specified edits
- Quick factual questions

Thinking (extended reasoning)

The model reasons step-by-step *before* acting. Slower, more tokens, fewer mistakes.

- Debugging stubborn errors
- Specification decisions
- Planning a multi-step change

When in doubt

If the task involves a **judgment call** or Claude got it wrong once already, turn thinking on. For mechanical work, leave it off.

Dialing the Effort

Trigger thinking with plain words. Adding a thinking cue to your prompt tells Claude how much reasoning budget to spend — more words, more thinking:

What you type	Effect
(nothing)	Answers directly
"`think`"	A little reasoning first
"`think hard`"	More careful reasoning
"`ultrathink`"	Maximum reasoning budget

The effort setting. Both **Sonnet** and **Opus** have an **effort** setting you can pick with `/model`, from least to most thorough:

`low` → `medium` → `high` → `extra high` → `max` → `ultracode`

Higher effort = more thorough, slower, costlier. Think of it as the same knob, set once for the session rather than per-prompt.

Thinking is not free. It spends tokens and time — reach for it when correctness matters, not by reflex.

Getting Files In: Drag and Drop

You don't have to describe a file in words — you can hand it straight to Claude.

Hold `Shift` and drop into the chat panel (in VS Code) to add something as context:

- A **data file** (`.csv`, `.xlsx`) so Claude can see the columns
- A **figure or PDF** — a paper, a table, a chart to reproduce
- A **screenshot** of an error message or a plot that looks wrong

Two ways to do it

- **Shift+drag** a file from the VS Code Explorer (or your Finder/desktop) onto the chat box — holding `Shift` is what drops it in as context instead of opening it
- **Paste** an image directly with `Cmd+V` / `Ctrl+V` — great for screenshots

A screenshot of a broken plot plus "why does this look wrong?" is often faster than describing it.

Cost Management

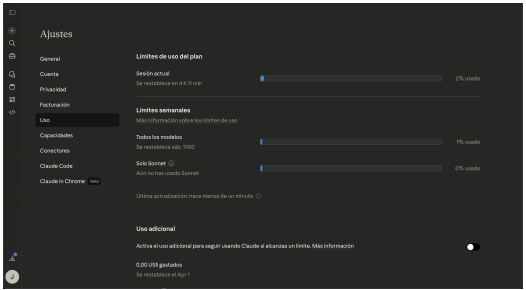
A Real Constraint

On the \$20/month plan, you can burn through your allocation in 30–45 minutes of active use if you are not strategic.

Principles:

1. Not everything needs Claude Code. Small edits are faster by hand
2. A well-written CLAUDE.md reduces back-and-forth (= fewer tokens)
3. If Claude Code is spinning on a wrong approach, **intervene early**
4. Batch related tasks into one well-scoped request
5. Plan mode adds tokens but prevents costly wrong approaches

Usage Limits and Model Cost



How limits work:

- Token allowance resets on a **5-hour rolling window**
- A separate **weekly cap** limits total usage
- When you hit either limit, Claude Code pauses until the window resets

Which models burn tokens fastest:

- **Opus** uses $\sim 5\times$ more tokens per task than Haiku
- **Sonnet** is the middle ground
- **Haiku** stretches your budget the furthest

Match the model to the task.

Don't use Opus to rename variables.

Where Claude Looks for Context

Claude Code reads instructions from **several levels** when it starts a session. Anything it finds is loaded into the context window before your first message.

- **User level** — `~/.claude/CLAUDE.md`
  Applies to *every* project on your machine. Personal preferences, defaults, things you want Claude to know about you regardless of what you are working on.
- **Project level** — `CLAUDE.md` in the project root
  Checked into git, shared with collaborators. Project conventions, data structure, constraints.
- **Subfolder level** — `CLAUDE.md` inside a subdirectory
  Loaded when Claude works in that subtree. Useful for scoped rules (e.g. a `cleaning/` folder with its own conventions).

How Claude Reads the Hierarchy

When Claude Code starts, it walks from the user level down to your working directory and loads **every applicable CLAUDE.md** it finds. They are concatenated into context, with more specific (deeper) files appearing later — so they take precedence when guidance conflicts.

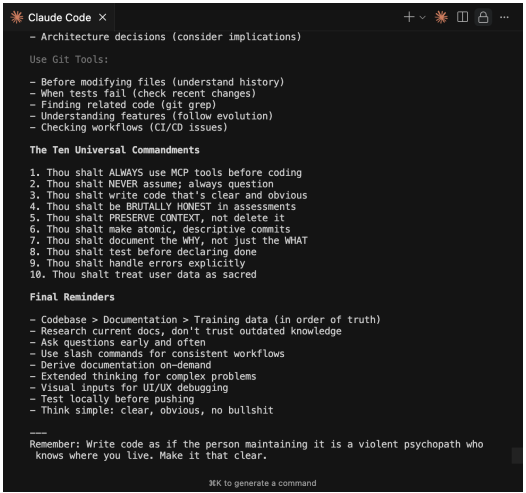
This hierarchy is everywhere

The same user → project → subfolder pattern shows up across the power-user sessions:

- **Skills** — defined at the user or project level, loaded on demand
- **Hooks** — configured per user or per project
- **MCP servers & permissions** — scoped per user or per project
- **Subagents** — registered at the user or project level

Learn the hierarchy once and it tells you where every other piece of configuration belongs.

CLAUDE.md: Your Lab Notebook for the Agent



Credit: u/rishabhsonker

Plain text files at the **user**, **project**, or **subfolder** level that Claude Code reads **automatically at the start of every conversation**.

They encode your conventions, definitions, and constraints so you do not have to repeat them each time.

The single most effective way to prevent errors before they happen.

Note: This does not guarantee Claude Code will always follow these instructions, but it significantly increases the chances.

What Belongs in CLAUDE.md

In the project-level CLAUDE.md, include:

- Project description & research question
- Data structure: files, key variables, units, expected ranges
- Coding conventions
- Constraints: "never drop observations without logging"
- "All merges must report row counts"
- What Claude Code should **not** do

Keep *user-level* CLAUDE.md for things that apply across projects; use a *subfolder* CLAUDE.md only when a directory needs its own rules.

How it interacts with the context window

Every applicable CLAUDE.md is loaded before your first message and stays "visible"—but it takes up space, so keep each one focused and concise.

CLAUDE.md Example

```
# CLAUDE.md - County Migration Project

## Research Question
Effect of historical migration on county-level outcomes.

## Data
- data/raw/census_panel.csv: county-year panel, 1920-2020
- Key vars: fips (5-digit), year, population, migration_share
- All monetary values in 2020 USD

## Constraints
- Never drop observations without logging count and reason
- All merges must report pre- and post-merge row counts
- Always use robust standard errors
- Do not choose model specifications
- Do not interpret coefficients
```

Exercise: Build the County Panel From Scratch

In Module 1 you ran a pre-built script. Now **you** build it.

Same data, fresh start. Work in pairs using plan mode.

1. Write a `CLAUDE.md` for the project (use the template)
2. Switch Claude Code to **plan mode**
3. Ask Claude Code to build a county-year panel that merges election returns with IPUMS census demographics
4. **Before approving the plan**, you will need to:
   - Read the codebooks to understand variable codings
     (What does `EDUC` $\geq$ 10 mean? What does `RACE == 1` code?)
   - Decide how to define key variables
     (Two-party vote share? College = bachelor's or higher?)
   - Specify the merge: which keys, which join type, expected row counts
5. Review the plan, reject or revise as needed, then approve
6. Run the script and verify the output

Evaluating the Plan

When Claude Code proposes its plan, ask yourself:

- Is it reading the **codebooks** or just guessing at variable meanings?
- How is it coding `EDUC`, `RACE`, `HISPAN`?
  Do those match *your* definitions?
- What join type is it using? Inner, left, or outer?
  What happens to counties that appear in one dataset but not the other?
- Is it filtering `mode == "TOTAL"` for election data?
- Does it handle `COUNTYFIP == 0` (unidentified counties)?

This is where your domain knowledge matters.
Claude Code can write the code, but you decide what the variables mean.

Debrief

- What variable coding decisions did you make? Did Claude Code's default match yours?
- Did anyone reject or modify the plan? What did you change?
- How did your panel compare to the Module 1 version?

The Lesson

Writing a CLAUDE.md forces you to be explicit about conventions you have kept implicit.

Reviewing plans forces you to think about approach *before* you see output.

Domain knowledge is key for preventing errors.

Module 2 Summary

1. Claude Code works with **files**, not interactive sessions
2. The **context window** limits what it can hold, so keep tasks focused
3. **Plan mode** is the default for research work
4. **CLAUDE.md** is your most powerful tool for preventing errors
5. **Cost management** is a core skill, not an afterthought

Next: Module 3, Automating Research Tasks